Mobile Security and Forensics

Practical Laboratory Exercises

Level 400 Undergraduate

Open-Source Tools Only | Real-World Scenarios

Scope note: These exercises are intended strictly for authorized, controlled academic environments. Students must only test devices, applications, and systems they own or are explicitly authorized to assess. Unauthorized testing on third-party or production systems may violate law and institutional policy.

Contents

Lab 1 — Forensic Recovery of Deleted Data from SQLite Databases
Lab 2 — Android Backup Analysis: Extracting Data from Non-Rooted Devices
Lab 3 — Exploiting Vulnerable Android Content Providers
Lab 4 — Dynamic Instrumentation with Frida: Bypassing Root Detection
Lab 5 — SSL Certificate Pinning Bypass for Mobile App Analysis
Lab 6 — Static and Dynamic Malware Analysis of Android Applications
Lab 7 — Android Security Assessment with Drozer Framework
Appendix — Common Issues and Troubleshooting

Lab 1: Forensic Recovery of Deleted Data from SQLite Databases

Scenario

A suspect's Android device has been seized in a drug trafficking investigation. The suspect claims they deleted all incriminating WhatsApp messages before the device was seized. You have obtained a file system extraction and need to recover deleted messages from the SQLite databases.

Learning Objectives

- Understand the internal structure of SQLite databases (headers, B-tree pages, freelist)
- Recover deleted records from SQLite databases using open-source tools
- Differentiate between active and recoverable deleted data
- Document forensic findings in a legally defensible manner

Tools Required

- DB Browser for SQLite (https://sqlitebrowser.org/) — GUI tool for viewing and editing SQLite databases
- Undark (https://github.com/CrackerCat/undark) — SQLite recovery tool for deleted data
- Hex editor (HxD for Windows or bless for Linux)
- SQLite3 command-line tool (pre-installed with most Linux distributions)

Setup Instructions

Step 1: Environment Preparation

```
# Install DB Browser for SQLite (Ubuntu/Debian)
sudo apt-get install sqlitebrowser

# Download Undark
git clone https://github.com/CrackerCat/undark.git
cd undark
make
```

Step 2: Create a Test SQLite Database

```
-- Create a test database simulating a messaging app
CREATE TABLE messages (
    _id INTEGER PRIMARY KEY AUTOINCREMENT,
    sender TEXT,
    recipient TEXT,
    message TEXT,
    timestamp INTEGER,
    deleted INTEGER DEFAULT 0
);

-- Insert sample data
INSERT INTO messages (sender, recipient, message, timestamp) VALUES
('alice', 'bob', 'Package delivered at the usual spot', 1700000000),
('bob', 'alice', 'Payment confirmed, see you tomorrow', 1700000100),
```

```
('alice', 'bob', 'The product is ready for pickup', 1700000200);

-- Simulate deletion
DELETE FROM messages WHERE message LIKE '%product%';
```

Investigation Procedure

Part A: Initial Examination with DB Browser

1. Open the database file in DB Browser for SQLite
2. Navigate to the "Browse Data" tab
3. Observe that the deleted message is no longer visible
4. Note the total number of records displayed

Part B: Manual Hex Analysis

1. Open the database file in a hex editor
2. Locate the SQLite header (offset 0): SQLite format 3
3. Identify the page size at offset 16 (2 bytes, big-endian)
4. Find the freelist page number at offset 32 (4 bytes)
5. Navigate to the freelist trunk page and identify leaf pages containing deleted data

Part C: Automated Recovery with Undark

```
# Run Undark against the database
./undark messages.db > recovered_data.txt

# Examine the output
cat recovered_data.txt
```

Part D: Forensic Documentation

Create a report documenting:

- Database file hash (SHA-256)
- Page size identified
- Number of freelist pages found
- Recovered records with timestamps
- Methodology used for recovery

Lab 2: Android Backup Analysis — Extracting Data from Non-Rooted Devices

Scenario

A corporate investigation requires examination of a company-owned Android device. The device is not rooted and cannot be modified. You need to extract forensic data using the Android Backup API without altering the device's security state.

Learning Objectives

- Perform ADB backup of Android applications
- Extract and analyze backup files using Android Backup Extractor
- Identify insecure data storage practices in Android applications
- Understand the forensic value of application backups

Tools Required

- Android SDK Platform-Tools (ADB) — https://developer.android.com/studio/releases/platform-tools
- Android Backup Extractor (ABE) — https://sourceforge.net/projects/adbextractor/
- 7-Zip or tar for extraction
- Java Runtime Environment (JRE)

Setup Instructions

Step 1: Install Required Tools

```
# Install ADB
sudo apt-get install android-tools-adb

# Download ABE
wget https://sourceforge.net/projects/adbextractor/files/abe.jar/download -O abe.jar
```

Step 2: Enable USB Debugging on the Device

1. Navigate to Settings → About Phone
2. Tap "Build Number" 7 times to enable Developer Options
3. Go to Settings → Developer Options
4. Enable "USB Debugging"
5. Connect the device and accept the RSA fingerprint prompt

Investigation Procedure

Part A: Create a Forensic Backup

```
# Verify device connection
adb devices
```

```
# Create a backup of the target application (replace with actual package name)
adb backup -f backup.ab -noapk com.example.targetapp

# User must confirm backup on the device screen
```

Part B: Extract the Backup File

```
# Convert .ab to .tar format
java -jar abe.jar unpack backup.ab backup.tar

# Extract the tar file
tar -xf backup.tar -C extracted/
# OR on Windows: Use 7-Zip to extract
```

Part C: Analyze Extracted Data

1. Navigate to the extracted directory structure
2. Examine the following key locations: apps/[package_name]/sp/ (shared preferences, XML files); apps/[package_name]/db/ (SQLite databases); apps/[package_name]/f/ (internal storage files); apps/[package_name]/manifest/ (AndroidManifest.xml)
3. Open any shared preferences XML files in a text editor
4. Open SQLite databases in DB Browser for SQLite

Part D: Security Assessment

Using the extracted data, identify the following:

- Sensitive data stored in plaintext (passwords, tokens, PII)
- Insecure data storage practices
- Application permissions and their justification

Lab 3: Exploiting Vulnerable Android Content Providers

Scenario

A banking application has been reported to have a vulnerability in its content provider implementation. As a security analyst, you must identify and exploit the vulnerability to demonstrate the risk to the organization.

Learning Objectives

- Identify exported content providers in Android applications
- Perform SQL injection attacks against content providers
- Extract sensitive data through vulnerable content providers
- Understand the impact of improperly secured content providers

Tools Required

- VulnLab APK (https://github.com/anpa1200/Vulnerable-APK)
- Android SDK Platform-Tools (ADB)
- Android emulator or physical device

Setup Instructions

Step 1: Install the Vulnerable Application

```
# Download and install VulnLab
git clone https://github.com/anpa1200/Vulnerable-APK.git
cd Vulnerable-APK
adb install app/build/outputs/apk/debug/app-debug.apk
```

Step 2: Launch the Application

1. Open the VulnLab app on the device/emulator
2. Note the various vulnerability categories available

Investigation Procedure

Part A: Identifying Exported Content Providers

Use adb shell to list package information:

```
adb shell dumpsys package com.example.vulnlab | grep -A 10 "Provider"
```

Identify content provider URIs from the AndroidManifest.xml:

```
adb shell dumpsys package com.example.vulnlab | grep "authorities"
```

Part B: Querying Content Providers

Attempt to query the content provider using the content command:

```
adb shell content query --uri content://com.example.vulnlab.provider/notes/
```

Document the columns returned and the data structure.

Part C: SQL Injection Exploitation

Test for SQL injection with a single quote:

```
adb shell content query --uri content://com.example.vulnlab.provider/notes/ --where "_id=1'"
```

Observe the error message revealing the SQL query structure. Extract data using UNION-based injection:

```
adb shell content query --uri content://com.example.vulnlab.provider/notes/ --where "_id=1) UNION SELECT
```

Extract table names:

```
adb shell content query --uri content://com.example.vulnlab.provider/notes/ --where "_id=1) UNION SELECT
```

Part D: Data Exfiltration

Extract sensitive data from the vulnerable content provider and document the findings.

Lab 4: Dynamic Instrumentation with Frida — Bypassing Root Detection

Scenario

A malicious application detects when the device is rooted and refuses to run, preventing forensic analysis. You must use dynamic instrumentation to bypass the root detection and analyze the application's behavior.

Learning Objectives

- Set up Frida for dynamic instrumentation
- Identify root detection mechanisms in Android applications
- Write Frida scripts to bypass root detection
- Perform runtime manipulation of application behavior

Tools Required

- Frida (https://frida.re/) — Dynamic instrumentation toolkit
- Frida Server for Android
- Python with Frida bindings
- Vulnerable APK with root detection

Setup Instructions

Step 1: Install Frida on Workstation

```
# Install Frida via pip
pip install frida-tools

# Verify installation
frida --version
```

Step 2: Install Frida Server on Device

```
# Download appropriate Frida server for device architecture
wget https://github.com/frida/frida/releases/download/16.0.0/frida-server-16.0.0-android-arm64.xz
unxz frida-server-16.0.0-android-arm64.xz

# Push to device
adb push frida-server-16.0.0-android-arm64 /data/local/tmp/frida-server
adb shell chmod 755 /data/local/tmp/frida-server

# Start Frida server
adb shell su -c /data/local/tmp/frida-server &
```

Part B: Port Forwarding and Testing

```
# Forward ports
adb forward tcp:27042 tcp:27042
adb forward tcp:27043 tcp:27043

# Test connection
```

```
frida-ps -U
```

Investigation Procedure

Part A: Identify Root Detection Methods

Common root detection checks include:

- Checking for su binary
- Checking Build.TAGS for "test-keys"
- Checking for Superuser.apk
- Checking /system/app/Superuser.apk

Part B: Create a Bypass Script

```
// root_bypass.js
Java.perform(function() {
    console.log("[*] Hooking root detection methods...");

    // Hook Runtime.exec to prevent finding 'su'
    var Runtime = Java.use("java.lang.Runtime");
    Runtime.exec.overload('[Ljava.lang.String;').implementation = function(args) {
        console.log("[*] exec called with: " + args);
        for (var i = 0; i < args.length; i++) {
            if (args[i].indexOf("su") !== -1) {
                console.log("[*] Blocking su command");
                throw new Error("Command not found");
            }
        }
        return this.exec(args);
    };

    // Hook Build.TAGS check
    var Build = Java.use("android.os.Build");
    Build.TAGS.value = "release-keys";
    console.log("[*] Build.TAGS patched");

    // Hook File.exists for Superuser.apk
    var File = Java.use("java.io.File");
    File.exists.implementation = function() {
        var path = this.getAbsolutePath();
        if (path.indexOf("Superuser.apk") !== -1 ||
            path.indexOf("su") !== -1) {
            console.log("[*] Blocking file existence check: " + path);
            return false;
        }
        return this.exists();
    };
});
```

Part C: Execute the Bypass

```
# Attach to the target application
frida -U -f com.example.targetapp -l root_bypass.js --no-pause
```

Part D: Verify Bypass Success

1 Observe the application behavior after hooking

2. The application should now run despite being on a rooted device
3. Document the bypassed checks and their implementations

Lab 5: SSL Certificate Pinning Bypass for Mobile App Analysis

Scenario

During a penetration test of a financial mobile application, you need to intercept and analyze the application's network traffic. The application implements SSL certificate pinning to prevent man-in-the-middle attacks. You must bypass this protection to perform a proper security assessment.

Learning Objectives

- Understand SSL certificate pinning mechanisms
- Use Xposed framework for runtime modification
- Bypass SSL pinning using SSL Killer module
- Intercept and analyze application traffic

Tools Required

- Xposed Framework or LSPosed (https://github.com/LSPosed/LSPosed)
- SSL Killer module (https://github.com/Xposed-Modules-Repo/com.simo.ssl.killer)
- Burp Suite or mitmproxy for traffic interception
- Rooted Android device or emulator

Setup Instructions

Step 1: Install Xposed/LSPosed Framework

1. Install LSPosed on the device (requires root)
2. Reboot the device after installation

Step 2: Install SSL Killer

```
# Download SSL Killer APK
wget https://github.com/Xposed-Modules-Repo/com.simo.ssl.killer/releases/download/2-2.0_Beta/SSL.Killer.

# Install the APK
adb install SSL.Killer.v2.0_Beta.apk
```

Step 3: Enable the Module

1. Open LSPosed Manager
2. Navigate to Modules
3. Enable SSL Killer for the target application

Investigation Procedure

Part A: Configure Burp Suite Proxy

1. Start Burp Suite and configure proxy listener on port 8080
2. Configure the Android device to use Burp as proxy
3. Attempt to browse HTTPS sites — note the certificate errors

Part B: Activate SSL Killer

1. Launch the SSL Killer application
2. Select the target application from the list
3. Enable SSL pinning bypass for the application

Part C: Intercept Application Traffic

1. Launch the target application
2. Observe traffic in Burp Suite
3. Document the requests and responses

Part D: Analyze Intercepted Data

Identify sensitive data transmitted by the application:

- Authentication tokens
- Personal information
- Financial data
- Session identifiers

Lab 6: Static and Dynamic Malware Analysis of Android Applications

Scenario

A suspicious APK file has been found on a compromised corporate device. You must analyze the application to determine its behavior, identify any malicious functionality, and assess the potential impact on the organization.

Learning Objectives

- Perform static analysis of Android APK files
- Decompile and examine application code
- Identify malicious behavior patterns
- Perform dynamic analysis using sandbox environments
- Generate comprehensive malware analysis reports

Tools Required

- Apktool (https://ibotpeaches.github.io/Apktool/) — APK decompilation
- dex2jar (https://github.com/pxb1988/dex2jar) — DEX to JAR conversion
- JD-GUI (http://jd.benow.ca/) — Java decompiler
- MobSF (https://github.com/MobSF/Mobile-Security-Framework-MobSF) — Automated analysis
- Frida for dynamic analysis

Setup Instructions

Step 1: Install Static Analysis Tools

```
# Install Apktool
wget https://raw.githubusercontent.com/iBotPeaches/Apktool/master/scripts/linux/apktool
chmod +x apktool
sudo mv apktool /usr/local/bin/

# Download dex2jar
wget https://github.com/pxb1988/dex2jar/releases/download/2.4/dex-tools-2.4.zip
unzip dex-tools-2.4.zip

# Install MobSF
git clone https://github.com/MobSF/Mobile-Security-Framework-MobSF.git
cd Mobile-Security-Framework-MobSF
./setup.sh
```

Investigation Procedure

Part A: Static Analysis with Apktool

```
# Decompile the APK
apktool d suspicious.apk -o suspicious_decompiled
```

```
# Examine AndroidManifest.xml
cat suspicious_decompiled/AndroidManifest.xml

# Look for suspicious permissions
grep -E "READ_SMS|RECORD_AUDIO|CAMERA|READ_CONTACTS|SEND_SMS" suspicious_decompiled/AndroidManifest.xml
```

Part B: Decompile to Java Source

```
# Convert DEX to JAR
./dex2jar/d2j-dex2jar.sh suspicious.apk -o suspicious.jar

# Open in JD-GUI (GUI tool)
jd-gui suspicious.jar
```

Part C: Automated Static Analysis with MobSF

1. Launch MobSF: ./run.sh
2. Upload the APK file through the web interface
3. Review the generated report for: permissions analysis, code analysis, file analysis, and API calls

Part D: Dynamic Analysis with Frida and MobSF

1. Set up an Android emulator
2. Configure MobSF for dynamic analysis
3. Run the application and monitor: network traffic, system calls, file system modifications, and data exfiltration attempts

Lab 7: Android Security Assessment with Drozer Framework

Scenario

You are tasked with performing a comprehensive security assessment of an Android application. Using the Drozer framework, you must identify and exploit vulnerabilities in the application's components, including activities, services, content providers, and broadcast receivers.

Learning Objectives

- Install and configure Drozer for Android security testing
- Identify the attack surface of Android applications
- Exploit vulnerabilities in exported components
- Assess Inter-Process Communication (IPC) security
- Generate security assessment reports

Tools Required

- Drozer (https://github.com/mwrlabs/drozer) — Security assessment framework
- Vulnerable APK (Sieve or GoatDroid)
- Android SDK Platform-Tools (ADB)
- Python environment

Setup Instructions

Step 1: Install Drozer

```
# Install Drozer via pip
pip install drozer

# Download Drozer Agent APK
wget https://github.com/mwrlabs/drozer/releases/download/2.4.4/drozer-agent-2.4.4.apk

# Install Agent on device
adb install drozer-agent-2.4.4.apk
```

Step 2: Port Forwarding

```
# Forward Drozer port
adb forward tcp:31415 tcp:31415
```

Step 3: Launch Drozer

```
# Start Drozer console
drozer console connect
```

Investigation Procedure

Part A: Package Enumeration

```
# List all packages
dz> run app.package.list
```

```
# Find target package
dz> run app.package.list -f [target_app_name]
```

Part B: Attack Surface Identification

```
# Identify attack surface
dz> run app.package.attacksurface [package_name]

# Expected output:
# Attack Surface:
#  2 activities exported
#  1 broadcast receivers exported
#  1 content providers exported
#  1 services exported
#  is debuggable
```

Part C: Activity Exploitation

```
# List exported activities
dz> run app.activity.info -a [package_name]

# Launch vulnerable activity
dz> run app.activity.start --component [package_name] [component_name]
```

Part D: Content Provider Assessment

```
# Find accessible content providers
dz> run scanner.provider.finduris -a [package_name]

# Query content provider
dz> run app.provider.query [uri]

# Test for SQL injection
dz> run scanner.provider.injection -a [package_name]
```

Part E: Broadcast Receiver Exploitation

```
# List exported receivers
dz> run app.broadcast.info -a [package_name]

# Send malicious broadcast
dz> run app.broadcast.send --action [action_name] --component [package_name] [receiver_name] --extra str
```

Appendix: Common Issues and Troubleshooting

ADB Connection Issues

- Ensure USB debugging is enabled
- Accept RSA fingerprint prompt on device
- Check device drivers (Windows)
- Restart ADB server: adb kill-server && adb start-server

SQLite Database Issues

- Database may be encrypted (SQLCipher)
- File permissions may prevent access on non-rooted devices
- WAL files may contain additional data not in main database

Frida Issues

- Ensure Frida server version matches client version
- Check port forwarding: adb forward tcp:27042 tcp:27042
- Root access required for most Frida operations

Xposed/LSPosed Issues

- Device must be rooted
- Module must be activated for target application
- Reboot required after enabling modules

Drozer Issues

- Ensure agent is running on device
- Check port forwarding: adb forward tcp:31415 tcp:31415
- Application must be installed and running

Note: All tools used in these labs are open-source and freely available. Students should only perform these exercises on devices they own or have explicit permission to test. Unauthorized testing on production systems may violate laws and organizational policies.